Commit 1b602dd

Browse files

ActuallyNull committed on Aug 10

made an ecg_augmentor class for data augmentation and implemented it into the data processing pipeline

main

1 parent 32ee47e commit 1b602dd

Filter files...

- data_processing.py
- ecg_augmentor.py

2 files changed +71 -2 lines changed

Search within code

data_processing.py +9 -2

```
@@ -6,6 +6,8 @@
from scipy.signal import butter, filtfilt, resample
from imblearn.over_sampling import SMOTE
from collections import Counter
+ from ecg_augmentor import ECG_Augmentor
+

dataset_folders = ["mit-bih", "training2017/training2017"]
records = []
@@ -80,11 +82,16 @@ def load_physionet_dataset(physionet_path, reference_file_path, X, y):

def preprocess_dataset(physionet_path, reference_file_path, X, y, fs=300): #X and y should be empty lists

+     augmentor = ECG_Augmentor(fs=fs)
+
      X, y = load_physionet_dataset(physionet_path, reference_file_path, X, y)
      X_filtered = np.array([butter_bandpass_filter(x, fs=fs) for x in X])

-     #X_standardised = standardise_dataset(X_filtered)
-     X_processed = normalise_dataset(X_filtered) # since this should be a dcnn, no standardisation needed
+     # Augment each signal (single-signal augmentation)
+     X_augmented = np.array([augmentor.augment(x) for x in X_filtered])
+
+     #X_standardised = standardise_dataset(X_augmented)
+     X_processed = normalise_dataset(X_augmented) # since this should be a dcnn, no standardisation needed
      y_processed = y

      # Apply SMOTE to balance the dataset
```

ecg_augmentor.py +62

```
@@ -0,0 +1,62 @@
+ import numpy as np
+ from scipy.signal import resample
+
+ class ECG_Augmentor:
+     def __init__(self, fs=360):
+         self.fs = fs  # sampling frequency
+
+     def add_gaussian_noise(self, signal, noise_level=0.01):
+         noise = np.random.normal(0, noise_level * np.max(signal), size=signal.shape)
+         return signal + noise
+
+     def baseline_wander(self, signal, freq=0.3, amp=0.05):
+         t = np.arange(len(signal)) / self.fs
+         drift = amp * np.sin(2 * np.pi * freq * t)
+         return signal + drift
+
+     def time_shift(self, signal, shift_max=0.1):
+         shift = int(np.random.uniform(-shift_max, shift_max) * len(signal))
+         return np.roll(signal, shift)
+
+     def time_scale(self, signal, scale_range=(0.9, 1.1)):
+         scale = np.random.uniform(*scale_range)
+         new_len = int(len(signal) * scale)
+         scaled = resample(signal, new_len)
+         # Crop or pad to original length
+         if len(scaled) > len(signal):
+             return scaled[:len(signal)]
+         else:
+             return np.pad(scaled, (0, len(signal) - len(scaled)), mode='constant')
+
+     def amplitude_scale(self, signal, scale_range=(0.9, 1.1)):
+         scale = np.random.uniform(*scale_range)
+         return signal * scale
+
+     def amplitude_shift(self, signal, shift_range=(-0.1, 0.1)):
+         shift = np.random.uniform(*shift_range) * np.max(signal)
+         return signal + shift
+
+     def mixup(self, sig1, sig2, alpha=0.2):
+         lam = np.random.beta(alpha, alpha)
+         return lam * sig1 + (1 - lam) * sig2, lam
+
+     def augment(self, signal, mix_signal=None):
+         """Apply a random set of augmentations"""
+         aug_signal = signal.copy()
+
+         if np.random.rand() < 0.5:
+             aug_signal = self.add_gaussian_noise(aug_signal)
+         if np.random.rand() < 0.3:
+             aug_signal = self.baseline_wander(aug_signal)
+         if np.random.rand() < 0.5:
+             aug_signal = self.time_shift(aug_signal)
+         if np.random.rand() < 0.4:
+             aug_signal = self.time_scale(aug_signal)
+         if np.random.rand() < 0.5:
+             aug_signal = self.amplitude_scale(aug_signal)
+         if np.random.rand() < 0.3:
+             aug_signal = self.amplitude_shift(aug_signal)
+         if mix_signal is not None and np.random.rand() < 0.3:
+             aug_signal, _ = self.mixup(aug_signal, mix_signal)
+
+         return aug_signal
```

Comments 0

Please sign in to comment.